

Architecture du projet

Documentation finale — application SmartFridge (réfrigérateur connecté)
Frontend React / TypeScript · Backend Node.js / Express · MongoDB / Mongoose
Basée uniquement sur le code réellement présent dans le dépôt

1. Vue d'ensemble

SmartFridge (ROCT) est une application de gestion de réfrigérateur : l'utilisateur enregistre ses produits, surveille leurs dates d'expiration, reçoit des notifications automatiques, trace ses consommations (avec leur valeur monétaire) et visualise ses pertes alimentaires. Un espace administrateur permet de gérer les utilisateurs et les catégories.



2. Technologies utilisées

Couche	Technologies	Usage
Frontend	React 19, TypeScript 6, Vite 8, react-router-dom 7, html5-qrcode, oxlint	SPA, routing, lecture de code-barres via caméra, lint
État frontend	Hooks React (useState / useEffect) + localStorage	État local par page ; token JWT conservé dans le navigateur (pas de store global)
Backend	Node.js, Express 5, dotenv, cors	API REST
Authentification	jsonwebtoken (JWT), bcryptjs	JWT expirant en 1 jour ; mots de passe hachés (10 rounds)
Validation & tâches	Zod, node-cron	Validation des requêtes ; job quotidien d'expiration

Base de données	MongoDB, Mongoose 9	Modèles & relations (ODM)
API externe	Open Food Facts API v3 (world.openfoodfacts.org)	Renseigner un produit à partir d'un code-barres

3. Frontend

3.1 Entrée et structure

- `src/main.tsx` — point d'entrée (React StrictMode) ; `src/App.tsx` — définition des routes.
- Routage (react-router-dom 7) : pages publiques *sans* sidebar (Login, Register) ; pages d'application *avec* sidebar via la route `Layout`.

Route	Page	Layout
/ et /login	Login	Sans sidebar
/register	Register	Sans sidebar
/dashboard	Dashboard	Avec sidebar
/products	Products	Avec sidebar
/products/:id	ProductDetails	Avec sidebar
/products/:id/edit	EditProduct	Avec sidebar — accessible mais non reliée par l'interface À vérifier
/notifications	Notifications	Avec sidebar
/profile	Profile	Avec sidebar

3.2 Composants

- `components/Layout.tsx` — structure `app-layout` : Sidebar + zone de contenu (`<Outlet/>`). Il détient l'état `collapsed` de la sidebar.
- `components/Sidebar.tsx` — logo ROCT, bouton toggle, navigation (Dashboard, Products, Notifications, Profile), bouton de déconnexion. Se replie sur simple icône.

3.3 Pages (fonctionnalités UI)

Page	Rôle
Login	Connexion (email/mot de passe), stockage du JWT, redirection vers /dashboard ; lien vers /register.
Register	Inscription (nom, email, mot de passe, confirmation) avec validations ; lien vers /login ; notification de bienvenue créée côté serveur.
Dashboard	Statistiques (total, valides, bientôt expirés, expirés), liste Expiring Soon / Expired, insights mensuels (consommation & pertes), vue des produits avec recherche et filtre par catégorie.
Products	Ajout de produit (manuel ou par scan caméra → Open Food Facts), liste avec recherche + filtre, lien « Voir les détails ».
ProductDetails	Fiche produit, statut d'expiration (jours restants / écoulés), consommation d'une quantité, modification inline, suppression.
Notifications	Liste des notifications (bienvenue / bientôt expiré / expiré), filtre par date, marquer comme lue, compteur de non-lues.

Profile	Profil (nom, email, rôle), modification du profil (mot de passe requis pour changer l'email), déconnexion ; section administration si rôle admin.
---------	---

3.4 Services (communication API)

Tous les services utilisent `fetch` vers `http://localhost:3001/api/*` et envoient le token JWT (www localStorage) dans l'en-tête `Authorization: Bearer <token>`. Les erreurs sont propagées via des exceptions avec le message serveur.

Service	Fonctions	API appelée
authService	registerUser, loginUser, getProfile, updateProfile, logoutUser	/api/auth/*
productService	getProducts, getProductById, createProduct, updateProduct, deleteProduct, consumeProduct, getProductByBarcode	/api/products/*
categoryService	getCategories, createCategory, updateCategory, deleteCategory	/api/categories/*
notificationService	getNotifications(date?), markNotificationAsRead	/api/notifications/*
insightsService	getMonthlyConsumption, getMonthlyLoss	/api/consumptions/monthly · /api/losses/monthly
adminService	getAdminStatistics, getAdminUsers, toggleUserStatus	/api/admin/*

3.5 Authentification côté frontend

- Le JWT renvoyé par `/api/auth/login` est stocké dans `localStorage` (« token »).
- Chaque requête authentifiée l'ajoute en en-tête `Authorization`.
- La déconnexion appelle `/api/auth/logout` puis supprime le token et renvoie vers la page de connexion.
- **À vérifier** — aucune garde de route frontend n'est présente : les pages d'application supposent le token valide (aucun composant `ProtectedRoute`).

4. Backend

4.1 Serveur (server.js)

- Express + `cors` + `express.json()` ; route racine de santé « SmartFridge Backend is running! ».
- 7 groupes de routes montés : `/api/auth` , `/api/products` , `/api/notifications` , `/api/categories` , `/api/admin` , `/api/consumptions` , `/api/losses` .
- Connexion MongoDB (`config/db.js` , via `process.env.MONGO_URI`), écoute sur le port **3001**.
- Lancement du contrôle d'expiration au démarrage (`checkExpiringProducts()`).

4.2 Middleware

Middleware	Rôle
<code>authMiddleware</code>	Extrait le Bearer token, le vérifie avec <code>JWT_SECRET</code> , expose <code>req.user = { id, role }</code> . Sinon 401.
<code>adminMiddleware</code>	Vérifie <code>req.user.role === "admin"</code> . Sinon 403.
<code>validationMiddleware</code>	Exécute un schéma Zod sur le corps de la requête ; renvoie 400 avec la liste des champs en erreur.

4.3 Validation (Zod)

- `authValidation.js` — *registerSchema* (nom 2–50, email, mot de passe 6–100), *loginSchema* (email, mot de passe non vide). `updateProfileSchema` est défini mais **non appliqué** à la route `PUT /api/auth/profile` [À vérifier](#) .
- `productValidation.js` — *productSchema* : nom ≥ 2 , catégorie (chaîne), dates valides, `expirationAlertDays  $\geq 0$` , `quantity  $\geq 1$` , `unit` $\in$ {piece, kg, g, l, ml}, `price  $\geq 0$` .

4.4 Services & tâche planifiée

- `services/openFoodFactsService.js` — interroge `world.openfoodfacts.org/api/v3/product/{code}` ; renvoie nom, quantité, unité, image et une catégorie mappée (fruits, légumes, laitiers, viandes, boissons, surgelés, conserves, « autre ») vers des identifiants MongoDB. [À vérifier](#) — ces identifiants sont codés en dur et présupposent des catégories pré-existantes en base (aucun script de seeds dans le dépôt).
- `jobs/expirationNotification.js` — `node-cron` chaque jour à 00:00 (et à chaque démarrage) : pour chaque produit (quantité > 0), crée/met à jour une notification « bientôt expiré » (réitération toutes les 24 h, arrêt une fois lue) ou une notification unique « expiré ».

4.5 Logique métier clé

- Statut d'un produit** (calculé dans les controllers) : `joursRestants = [(expiration - aujourd'hui)/jour]` → `expire` si < 0 , `bientot_expire` si $\leq$ `expirationAlertDays` , sinon `valide` . Le statut est renvoyé calculé et aussi persisté.
- Consommation** : prix unitaire = `price / initialQuantity` ; montant consommé = prix unitaire $\times$ quantité ; décrémente `quantity` et écrit une entrée `Consumption` (traçabilité).
- Pertes mensuelles** : somme, sur les produits expirés du mois, de `(price / initialQuantity)  $\times$  quantity` .
- Consommation mensuelle** : somme des montants enregistrés par jour de consommation.
- Inscription** : création de l'utilisateur (mot de passe haché, bcrypt) + notification de bienvenue.
- Changement d'email** : mot de passe actuel requis et vérifié.
- Compte désactivé** : refus de connexion (403 « Account is deactivated »).

5. Base de données (Modèles Mongoose)

Modèle	Champs principaux	Rôle & relations
--------	-------------------	------------------

User	name · email (unique) · password (hash) · isActive (défaut true) · role (user/admin, défaut user)	Comptes et authentification. Référencé par Product, Notification, Consumption.
Category	name (unique)	Catégories de produits. Référencée par Product (populate selon le nom).
Product	name · category (ref Category) · purchaseDate · expirationDate · expirationAlertDays · quantity · initialQuantity · unit · price · status · user (ref User)	Cœur du domaine. Relation <i>User</i> → <i>Products</i> (propriété), <i>Category</i> → <i>Product</i> .
Notification	user (ref) · product (ref, optionnelle) · message · type (bientot_expire / expire / welcome) · isRead · lastSentAt	Alertes de l'utilisateur. Reliée à l'utilisateur et éventuellement au produit concerné.
Consumption	product (ref) · user (ref) · quantity · amount · date	Traçabilité des consommations (montant en DH). Reliée à Product et User.

6. API REST — routes disponibles

Auth — /api/auth

Méthode	Route	Auth	Description
POST	/register	—	Inscription (Zod) → utilisateur + notification de bienvenue (201)
POST	/login	—	Connexion → { token, user } (durée 1 jour) ; refus si compte désactivé
GET	/profile	Bearer	Profil courant (sans mot de passe)
PUT	/profile	Bearer	Modification nom/email (mot de passe requis pour l'email)
POST	/logout	Bearer	Confirme la déconnexion (suppression du token côté client)

Products — /api/products

Méthode	Route	Auth	Description
GET	/	Bearer	Produits de l'utilisateur avec statut calculé + catégorie peuplée
GET	/barcode/:barcode	Bearer	Recherche produit via Open Food Facts (nom, quantité, unité, image, catégorie)
GET	/:id	Bearer	Un produit (scopé utilisateur) avec statut
POST	/	Bearer + validation	Création (initialQuantity = quantity)
PUT	/:id	Bearer + validation	Mise à jour complète (scopée utilisateur)
DELETE	/:id	Bearer	Suppression (scopée utilisateur)
PATCH	/:id/consume	Bearer	Consommation : décrémente la quantité, enregistre une entrée Consumption

Categories — /api/categories

Méthode	Route	Auth	Description
GET	/	Bearer	Toutes les catégories (tri par nom)

POST	/	Bearer + admin	Créer une catégorie
PUT	/:id	Bearer + admin	Renommer
DELETE	/:id	Bearer + admin	Supprimer

Notifications — /api/notifications

Méthode	Route	Auth	Description
GET	/?date=YYYY-MM-DD	Bearer	Notifications de l'utilisateur (filtre date optionnel, tri décroissant)
PATCH	/:id/read	Bearer	Marquer comme lue

Admin — /api/admin

Méthode	Route	Auth	Description
GET	/statistics	Bearer + admin	{ totalUsers, totalProducts, expiredProducts }
GET	/users	Bearer + admin	Tous les utilisateurs (sans mot de passe)
PUT	/users/:id/status	Bearer + admin	Activer / désactiver un compte

Statistiques mensuelles

Méthode	Route	Auth	Description
GET	/api/consumptions/monthly?year=&month=	Bearer	Total consommé (somme des montants en DH) pour le mois
GET	/api/losses/monthly?year=&month=	Bearer	Total des pertes (produits déjà expirés) pour le mois

7. Fonctionnalités ↔ Backend

Fonctionnalité (frontend)	Appels back-end	Modules impliqués
Inscription / Connexion / Déconnexion	POST /auth/register · /auth/login · /auth/logout	authController, bcrypt, jwt, Notification (welcome)
Profil & administration du profil	GET/PUT /auth/profile	authController
Ajout produit (manuel)	POST /products	productController, productValidation, Product
Ajout produit (scan)	GET /products/barcode/:code	openFoodFactsService, productController
Liste / recherche / filtre produits	GET /products · catégories locales	productController
Détails / modification inline / suppression	GET /:id · PUT /:id · DELETE /:id	productController
Consommation d'un produit	PATCH /products/:id/consume	productController, Consumption
Statuts & alertes d'expiration	calculé dans GET/POST/PUT products	productController (jours restants)
Notifications (bienvenue / expiration)	GET /notifications · PATCH /:id/read	notificationController, job expirationNotification
Dashboard — statistiques & alertes	GET /products (+ calcul local)	productController
Dashboard — insights mensuels	GET /consumptions/monthly · /losses/monthly	consumptionController, lossController
Catégories	GET /categories ; POST/PUT/DELETE (admin)	categoryController
Espace administrateur	GET /admin/statistics · /admin/users · PUT /admin/users/:id/status	adminController, adminMiddleware

Points de vigilance (fidélité au code)

- La page `EditProduct` reste routée à `/products/:id/edit`, mais le bouton « Modifier » de `ProductDetails` ouvre désormais un formulaire inline : la page d'édition n'est plus reliée par l'interface À vérifier.
- Sur le Dashboard, la carte « Consommation » affiche la *valeur* consommée (somme des montants, en DH) sans suffixe monétaire, alors que « Food Waste » affiche « DH » À vérifier.
- Mapping de catégories d'Open Food Facts vers des ObjectId MongoDB codés en dur : dépend de catégories pré-enregistrées en base (aucun seed fourni) À vérifier.
- `updateProfileSchema` (Zod) défini mais non branché sur la route `PUT /api/auth/profile` À vérifier.
- Aucune garde de route frontend (les routes d'application supposent un token). À vérifier

ROCT · SmartFridge — Documentation d'architecture rédigée à partir du code source actuel du dépôt github.com/tourabikhadija/SmartFridge.